

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

И.Д. Свеженин

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Намерения, меню и работа с данными

по курсу: Кроссплатформенное программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1. Задание	2
2. Ход работы.....	3
3. Результат работы программы.....	5
4. Вывод.....	8
5. Листинг с кодом программы.....	9

1. Задание

В рамках лабораторной работы необходимо было создать приложение для получения и отображения данных о гонщиках и их автомобилях с открытого API, реализовать навигацию между вкладками с использованием BottomNavigationView, оформить каждый блок информации в отдельный фрагмент, обеспечить интерактивный выбор гонщика через список с RecyclerView, корректно отображать данные выбранного гонщика и автомобиля, а также предусмотреть обработку отсутствия исключений.

2. Ход работы

В процессе выполнения лабораторной работы приложение было построено вокруг трех фрагментов и одного активити. Основной функционал реализован в MainActivity, где создаются экземпляры фрагментов и организуется навигация с помощью BottomNavigationView. Для работы с данными гонщиков используется метод loadDriversList, который с помощью корутин и HttpURLConnection отправляет GET-запрос к API, получает JSON и заполняет список гонщиков. Выбор гонщика осуществляется через RecyclerView, где адаптер DriversAdapter обрабатывает нажатия и вызывает метод loadDriverInfo для получения подробной информации о выбранном спортсмене. Полученные данные отображаются в TextView внутри фрагмента, а текущий выбранный индекс гонщика сохраняется в переменной currentDriverIndex для корректного отображения информации при переключении вкладок.

Для отображения информации о машине гонщика используется метод loadCarInfo, который делает отдельный запрос к API с номером выбранного гонщика. Полученные параметры автомобиля, такие как скорость, обороты двигателя, передача, состояние тормоза и газа, обрабатываются и выводятся в TextView внутри фрагмента авто. При этом реализована проверка на наличие интернет-соединения, и в случае его отсутствия выводится соответствующее сообщение. Все сетевые операции выполняются в корутинах на фоне, чтобы не блокировать пользовательский интерфейс, а обновление данных на экране происходит через lifecycleScope.

Навигация между фрагментами реализована с помощью FragmentManager и методов replace и commit, что позволяет динамически подставлять нужный фрагмент в контейнер активити. RecyclerView используется для динамического отображения списка гонщиков с обработкой кликов, что обеспечивает интерактивное взаимодействие. TextView внутри каждого фрагмента автоматически обновляется при выборе гонщика или при переключении на вкладку автомобиля, используя вызовы suspend функций

loadDriverInfo и loadCarInfo. Такой подход обеспечивает модульность кода, асинхронную загрузку данных и корректное сохранение состояния выбранного гонщика при переключении между вкладками.

3. Результат работы программы

В качестве результата работы программы представлено пару фотографий интерфейса. На рисунках 1-2 показан пример работы программы.

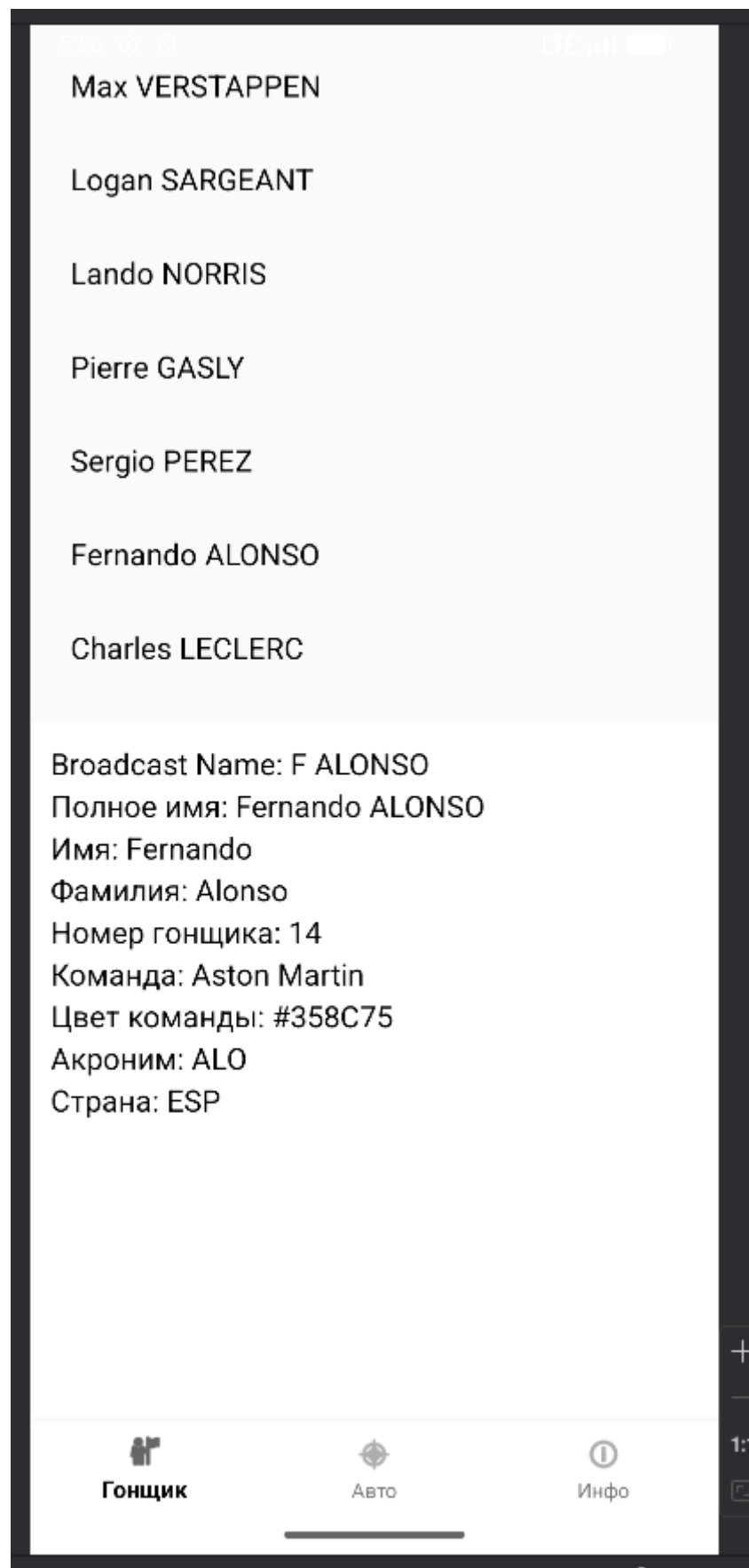


Рисунок 1 – Выбор гонщика

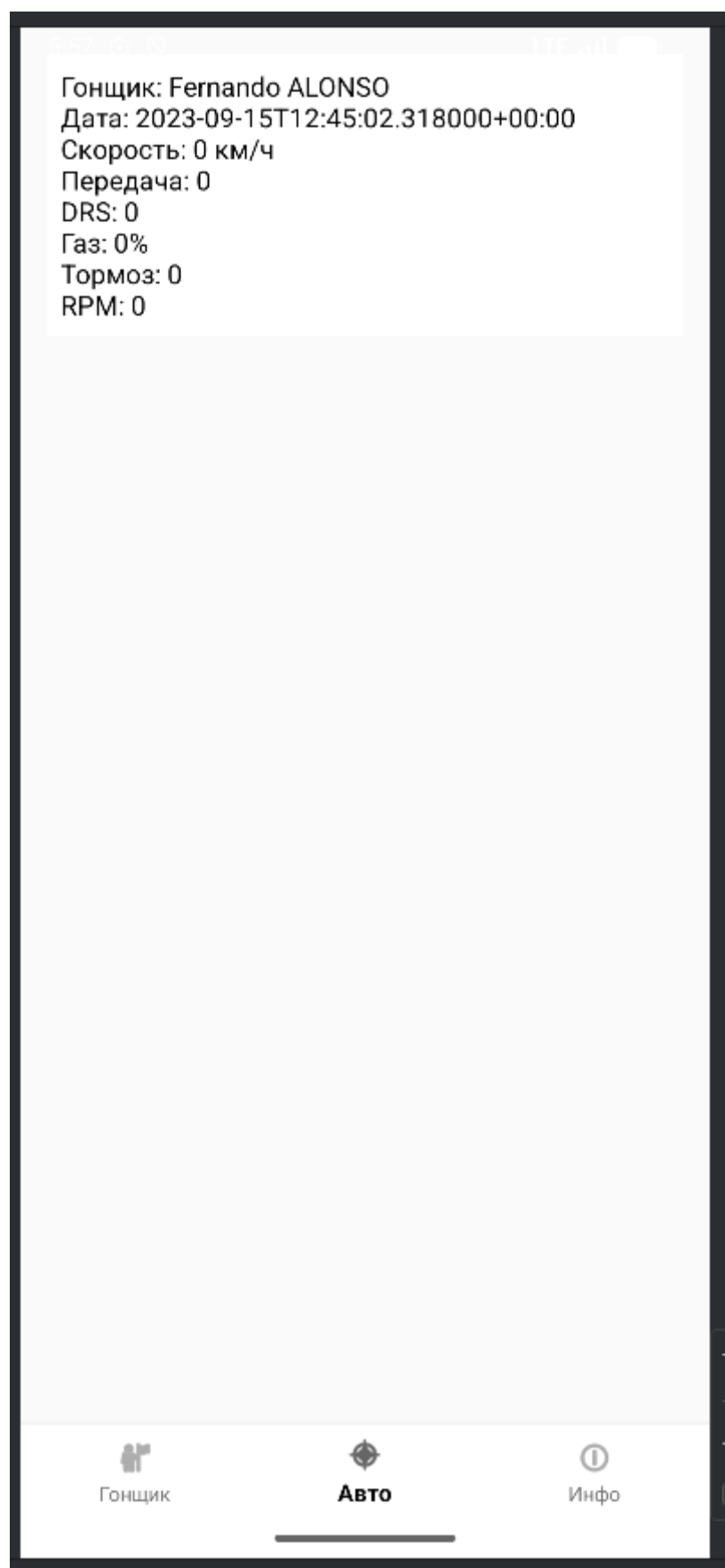


Рисунок 2 – Просмотр характеристики гонщика

4. Вывод

В результате выполнения лабораторной работы было разработано мобильное приложение, позволяющее получать и отображать актуальные данные о гонщиках Формулы 1 и их автомобилях с открытого API. Приложение реализует навигацию между двумя основными вкладками: информация о гонщике и данные автомобиля, при этом каждая вкладка оформлена в виде отдельного фрагмента, что облегчает управление интерфейсом и повышает читаемость кода.

Для отображения списка гонщиков использован компонент RecyclerView, обеспечивающий удобный выбор и обработку нажатий на элементы списка. При выборе гонщика приложение асинхронно загружает информацию через корутины и отображает её в текстовом блоке с корректным форматированием, включая имя, команду, страну, а также основные параметры автомобиля, такие как скорость, обороты двигателя, передача и состояние педалей.

Также реализована обработка отсутствия интернет-соединения: при попытке загрузки данных без подключения пользователь получает уведомление об ошибке, что повышает стабильность работы приложения. Использование корутин и разделение логики на фрагменты обеспечили плавность интерфейса, быстрый отклик на действия пользователя и простоту дальнейшей поддержки и расширения функционала.

5. Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:
#mainactivity.kt

```
package com.example.autotasks

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
import androidx.lifecycle.LifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import
com.google.android.material.bottomnavigation.BottomNavigationView
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import org.json.JSONArray
import java.net.HttpURLConnection
import java.net.URL

// MainActivity
class MainActivity : AppCompatActivity() {

    private val drivers = mutableListOf<Pair<String, Int>>() // Pair<fullName,
driver_number>
```

```

private var currentDriverIndex = 0

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val bottomNav =
findViewById<BottomNavigationView>(R.id.bottomNavigation)

    // Загружаем гонщиков
    lifecycleScope.launch {
        loadDriversList()
        if (drivers.isNotEmpty()) {
            currentDriverIndex = 0
        }
        supportFragmentManager.beginTransaction()
            .replace(R.id.fragmentContainer,
FragmentDriver.newInstance(drivers, ::loadDriverInfo))
            .commit()
    }

    bottomNav.setOnItemSelectedListener { item ->
        when (item.itemId) {
            R.id.menu_driver -> {
                supportFragmentManager.beginTransaction()
                    .replace(R.id.fragmentContainer,
FragmentDriver.newInstance(drivers, ::loadDriverInfo))
                    .commit()
                true
            }
        }
    }
}

```

```

        R.id.menu_car -> {
            supportFragmentManager.beginTransaction()
                .replace(R.id.fragmentContainer,
FragmentCar.newInstance(drivers[currentDriverIndex].second, ::loadCarInfo))
                .commit()
            true
        }
        else -> false
    }
}
}

```

```

private suspend fun loadDriversList() {
    withContext(Dispatchers.IO) {
        try {
            val url =
URL("https://api.openf1.org/v1/drivers?session_key=9158")
            val conn = url.openConnection() as HttpURLConnection
            conn.requestMethod = "GET"
            conn.connectTimeout = 5000
            conn.readTimeout = 5000

            if (conn.responseCode != 200) return@withContext

            val data = conn.inputStream.bufferedReader().readText()
            val json = JSONArray(data)
            drivers.clear()
            for (i in 0 until json.length()) {
                val obj = json.getJSONObject(i)
                val fullName = obj.optString("full_name")
            }
        }
    }
}

```

```

        val driverNum = obj.optInt("driver_number")
        drivers.add(Pair(fullName, driverNum))
    }
} catch (e: Exception) {
    e.printStackTrace()
}
}
}

```

```

private suspend fun loadDriverInfo(driverNumber: Int): String {
    return withContext(Dispatchers.IO) {
        try {
            val url =
                URL("https://api.openf1.org/v1/drivers?driver_number=$driverNumber&session_
                    key=9158")

            val conn = url.openConnection() as HttpURLConnection
            conn.requestMethod = "GET"
            conn.connectTimeout = 5000
            conn.readTimeout = 5000

            if (conn.responseCode != 200) return@withContext "Ошибка
                запроса: ${conn.responseCode}"

```

```

            val data = conn.inputStream.bufferedReader().readText()
            val json = JSONArray(data)
            if (json.length() == 0) return@withContext "Гонщик не найден"

```

```

            val obj = json.getJSONObject(0)
            val broadcastName = obj.optString("broadcast_name")
            val countryCode = obj.optString("country_code")

```

```

        val driverNum = obj.optInt("driver_number")
        val firstName = obj.optString("first_name")
        val fullName = obj.optString("full_name")
        val lastName = obj.optString("last_name")
        val teamName = obj.optString("team_name")
        val teamColour = obj.optString("team_colour")
        val nameAcronym = obj.optString("name_acronym")

        return@withContext """
            Broadcast Name: $broadcastName
            Полное имя: $fullName
            Имя: $firstName
            Фамилия: $lastName
            Номер гонщика: $driverNum
            Команда: $teamName
            Цвет команды: #$teamColour
            Акроним: $nameAcronym
            Страна: $countryCode
        """.trimIndent()
    } catch (e: Exception) {
        return@withContext "Ошибка: ${e.message}"
    }
}

private suspend fun loadCarInfo(driverNumber: Int): String {
    return withContext(Dispatchers.IO) {
        try {

```

```

        val url =
URL("https://api.openf1.org/v1/car_data?driver_number=$driverNumber&session
_key=9159")

        val conn = url.openConnection() as HttpURLConnection
        conn.requestMethod = "GET"
        conn.connectTimeout = 5000
        conn.readTimeout = 5000

        if (conn.responseCode != 200) return@withContext "Ошибка
запроса: ${conn.responseCode}"

        val data = conn.inputStream.bufferedReader().readText()
        val json = JSONArray(data)
        if (json.length() == 0) return@withContext "Данные автомобиля
не найдены"

        val obj = json.getJSONObject(0)
        val speed = obj.optInt("speed")
        val nGear = obj.optInt("n_gear")
        val drs = obj.optInt("drs")
        val throttle = obj.optInt("throttle")
        val brake = obj.optInt("brake")
        val rpm = obj.optInt("rpm")
        val date = obj.optString("date")

        return@withContext ""
        Дата: $date
        Скорость: $speed км/ч
        Передача: $nGear
        DRS: $drs

```

```

        Газ: $throttle%
        Тормоз: $brake
        RPM: $rpm
        """".trimIndent()
    } catch (e: Exception) {
        return@withContext "Ошибка: ${e.message}"
    }
}
}
}

// Fragment для гонщика
class FragmentDriver : Fragment() {

    private lateinit var drivers: List<Pair<String, Int>>
    private lateinit var loadDriverInfo: suspend (Int) -> String
    private var currentDriverIndex = 0

    companion object {
        fun newInstance(
            drivers: List<Pair<String, Int>>,
            loadDriverInfo: suspend (Int) -> String
        ): FragmentDriver {
            val fragment = FragmentDriver()
            fragment.drivers = drivers
            fragment.loadDriverInfo = loadDriverInfo
            return fragment
        }
    }
}

```

```

        override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?, savedInstanceState: Bundle?): View? {
            val view = inflater.inflate(R.layout.fragment_driver, container, false)
            val recyclerViewDrivers =
view.findViewById<RecyclerView>(R.id.recyclerDrivers)
            val textResult = view.findViewById<TextView>(R.id.textResult)

            recyclerViewDrivers.layoutManager =
LinearLayoutManager(requireContext())
            recyclerViewDrivers.adapter = DriversAdapter(drivers) { index ->
                currentDriverIndex = index
                lifecycleScope.launch {
                    textResult.text = loadDriverInfo(drivers[index].second)
                }
            }

            if (drivers.isNotEmpty()) {
                lifecycleScope.launch {
                    textResult.text =
loadDriverInfo(drivers[currentDriverIndex].second)
                }
            }

            return view
        }
    }

    // Fragment для авто
    class FragmentCar : Fragment() {

```

```

private var driverNumber: Int = 0

private lateinit var loadCarInfo: suspend (Int) -> String

companion object {
    fun newInstance(driverNumber: Int, loadCarInfo: suspend (Int) ->
String): FragmentCar {
        val fragment = FragmentCar()
        fragment.driverNumber = driverNumber
        fragment.loadCarInfo = loadCarInfo
        return fragment
    }
}

override fun onCreateView(inflater: LayoutInflater, container:
ViewGroup?, savedInstanceState: Bundle?): View? {
    val view = inflater.inflate(R.layout.fragment_car, container, false)
    val textCar = view.findViewById<TextView>(R.id.textCar)

    lifecycleScope.launch {
        textCar.text = loadCarInfo(driverNumber)
    }

    return view
}

// Адаптер RecyclerView для гонщиков
class DriversAdapter(
    private val drivers: List<Pair<String, Int>>,
    private val onItemClick: (Int) -> Unit

```

```

) : RecyclerView.Adapter<DriversAdapter.DriverViewHolder>() {

    inner class DriverViewHolder(view: View) :
RecyclerView.ViewHolder(view) {
        val textDriverName: TextView =
view.findViewById(R.id.textDriverName)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
DriverViewHolder {
        val view =
LayoutInflater.from(parent.context).inflate(R.layout.item_driver, parent, false)
        return DriverViewHolder(view)
    }

    override fun onBindViewHolder(holder: DriverViewHolder, position: Int)
{
        holder.textDriverName.text = drivers[position].first
        holder.itemView.setOnClickListener { onItemClick(position) }
    }

    override fun getItemCount(): Int = drivers.size
}

```